

Pendientes de probar — orden de prioridad

Generado: 2026-05-12 07:43

Total items: 21 Tests unitarios: 1064 pasando Planos enviados al CFIA: 6

Categoría	Cantidad	Significa
CRÍTICO	3	Eventos externos: esperar CFIA/Muni para validar
ALTA	5	Features ya implementadas; falta uso/validación live
MEDIA	7	Validación con datos reales en producción
BAJA	6	Experimental o requiere evento específico (raro)

Cómo usar este documento: los items están ordenados por urgencia. Items 1-3 son críticos para cerrar el ciclo end-to-end (esperan eventos externos). Items 4-8 son validación live de features ya construidas. 9-21 son secundarios.

#1 R2/R3 — ciclo de correcciones CFIA

CRÍTICO — cerrar ciclo

Depende de: Esperar respuesta CFIA (~7 días) de cualquiera de los 6 trámites enviados (1257831, 1257881, 1258052, 1258126).

Descripción: Cuando CFIA marca un plano como 'Defectuoso' o 'Calificación con observaciones', el bot debe: (a) detectar el cambio via scheduler apt-sync-estados, (b) descargar la minuta y la imagen-minuta del portal APT, (c) presentar las correcciones al operador via WhatsApp, (d) aplicar correcciones automáticas al DWG (si son de texto), (e) re-empacar y enviar R2, (f) repetir si CFIA devuelve R3.

Cómo probar:

- Esperar respuesta CFIA real de algún trámite enviado
- Verificar que apt-sync-estados detecta el cambio de estado
- Verificar que el bot baja la minuta y la presenta correctamente
- Probar correcciones automáticas vs correcciones que requieren intervención
- Subir R2 y validar que CFIA lo recibe correctamente

Componentes: src/agents/apt_agent.py · src/agents/minuta_agent.py · src/utils/dwg_corrector.py · scheduler tasks.py

Estado tests: ■ Existen tests con mocks. Falta validación end-to-end.

#2 Visado municipal (San Ramón)

CRÍTICO — cerrar ciclo

Depende de: Esperar respuesta de Muni San Ramón al plano FELIPE_TIOS (RDF-2026-004, segregación enviada al CFIA el 11-05-2026).

Descripción: Para segregaciones y reuniones de fincas, después de CFIA viene la municipalidad. Flujo esperado: (a) topógrafo envía el Google Form, (b) Muni responde por correo a mgamboa@sanramondigital.net con: APROBADO, RECHAZADO, o aviso de MOROSIDAD, (c) bot parsea el correo via IMAP, (d) actualiza estado, (e) si morosidad: solicita pago al cliente, recibe comprobante, lo reenvía a la Muni, (f) si aprobado: pasa a paso final.

Cómo probar:

- Enviar el Google Form de FELIPE_TIOS cuando CFIA acepte (~7 días)
- Esperar respuesta de Muni (puede ser horas o días)
- Verificar que el IMAP poll detecta el correo nuevo
- Validar el parser para cada caso (aprobado/rechazado/morosidad)
- Si morosidad: probar el sub-flujo de pago + reenvío

Componentes: src/agents/municipality_agent.py · IMAP poll del scheduler

Estado tests: ■ Tests con mocks. Falta validación end-to-end con correo real.

#3 Inscripción final + descarga plano firmado

CRÍTICO — cerrar ciclo

Depende de: CFIA marca el plano como 'Público e Inscrito' (típicamente 1-3 semanas después de la primera presentación).

Descripción: Cuando el plano queda Inscrito, el bot debe: (a) descargar el PDF firmado digitalmente del portal APT, (b) guardarlo en data/files/.../03_Catastrado/, (c) notificar al topógrafo que el plano está listo para entrega, (d) avisar al cliente cuando lo reciba físicamente, (e) marcar estado ENTREGADO.

Cómo probar:

- Esperar a que primer plano se marque Inscrito
- Verificar que apt-sync-estados detecta el cambio
- Verificar descarga automática del PDF inscrito
- Validar flujo de notificación al cliente
- Probar comando RECIBIR cuando el cliente confirma

Componentes: `src/agents/apt_agent.py` · `src/workflows/base_workflow.py` (paso INSCRITO)

Estado tests: ■ Tests existen. Falta validación con plano real inscrito.

#4 Auto-extracción con Claude Vision

ALTA — validar live

Depende de: Configurar ANTHROPIC_API_KEY en Windows Credential Manager.

Descripción: Probar el extractor completo end-to-end contra los 4 expedientes reales subidos (ROGRANJ, OMAR_2026, ROVUELT, FELIPE_TIOS). El bot debe leer plano.pdf + registro.png + entero.pdf, armar datos_apt con reglas de oficina aplicadas, y producir el mismo seed (o uno mejor) que se hizo manualmente.

Cómo probar:

- Configurar: `python -c "from src.core.credential_manager import CredentialManager; CredentialManager().set_secret('anthropic-api', 'sk-ant-...')"`
- Correr: `python tools/extraer_datos_apt.py RDF-2026-002`
- Comparar el JSON resultante con el `seed_datos_apt_omar.py` manual
- Repetir para los 4 expedientes (002, 003, 004 + uno nuevo)
- Validar advertencias y campos con baja confianza

Componentes: `src/utils/plano_vision_extractor.py` · `src/utils/seed_builder.py` · `tools/extraer_datos_apt.py`

Estado tests: ■ 33 tests con mocks de Anthropic API. Falta live test.

#5 OCR fallback (pypdf+regex) sin Vision

ALTA — validar live

Depende de: Configurar entorno sin ANTHROPIC_API_KEY para forzar el fallback.

Descripción: Cuando no hay API key o la API falla, el bot debe caer a pypdf+regex y extraer lo que pueda del cajetín. Validar qué campos recupera bien (área, protocolo, entero, identificador) y cuáles no (coordenadas, nombre propietario).

Cómo probar:

- Desconfigurar ANTHROPIC_API_KEY: `$env:ANTHROPIC_API_KEY=""`
- Correr: `python tools/extraer_datos_apt.py RDF-2026-002`
- Verificar que el extractor cae a pypdf
- Listar campos recuperados vs faltantes
- Validar que las advertencias señalan campos no extraídos

Componentes: `src/utils/plano_vision_extractor.py` (función `extract_cajetin_con_fallback`)

Estado tests: ■ 4 tests con mocks. Falta live test sin API key.

#6 Healthcheck server + watchdog Chrome

ALTA — validar live

Depende de: Bot corriendo en modo servicio Windows.

Descripción: El healthcheck server escucha en localhost:9223/health y devuelve JSON con el estado del sistema. El watchdog detecta si Chrome del bot crashea y lo relanza automáticamente.

Cómo probar:

- Instalar el servicio: `python -m src.service.windows_service install`
- Arrancarlo: `python -m src.service.windows_service start`
- Verificar healthcheck: `curl http://localhost:9223/health`
- Matar Chrome del bot manualmente: `taskkill /F /IM chrome.exe`
- Verificar que el watchdog lo relanza dentro de 10 segundos
- Conectar un monitor externo (ej. UptimeRobot) al endpoint

Componentes: `src/utils/healthcheck.py` · `src/service/windows_service.py`

Estado tests: ■ 14 tests con mocks. Falta validación en modo servicio real.

#7 Backup BD automático + restore

ALTA — validar live

Depende de: Dejar el scheduler corriendo durante 24h+.

Descripción: El scheduler hace backup a las 2:00 UTC diariamente (job 'db-backup' usando VACUUM INTO). Validar que crea el archivo, que la rotación funciona, y que el archivo es restaurable.

Cómo probar:

- Verificar que `data/backups/` existe y contiene `catastro-YYYY-MM-DD.db`
- Forzar backup manual: `python tools/backup_db.py`
- Listar: `python tools/backup_db.py --solo-listar`
- Probar restore: `cp data/backups/catastro-...db data/test-restore.db; sqlite3 data/test-restore.db '.tables'`
- Verificar que la rotación mantiene 30 backups

Componentes: `src/scheduler/tasks.py` (job db-backup) · `src/utils/backup_db.py` · `tools/backup_db.py`

Estado tests: ■ 13 tests del módulo `backup_db`. Falta validación en modo servicio.

#8 Dashboard + alertas proactivas

ALTA — validar live

Depende de: Acumular 20+ planos para tener datos significativos.

Descripción: El dashboard agrega métricas (estados, tipos, topógrafos, tiempos, discrepancias frecuentes, anomalías recientes). Las alertas proactivas detectan patrones (ej. 3+ anomalías en mismo contexto → posible cambio en APT).

Cómo probar:

- Correr con BD actual: `python tools/dashboard.py`
- Verificar que las métricas son consistentes con `listar_planos`

- Provocar anomalías deliberadas para activar alertas proactivas
- Probar formato JSON: `python tools/dashboard.py --json | jq .`
- Probar CSV import en Excel: `python tools/dashboard.py --csv > dashboard.csv`

Componentes: `src/utils/metricas.py` · `tools/dashboard.py`

Estado tests: ■ 16 tests. Falta validación con BD producción larga.

#9 Validación derrotero ZIP

MEDIA — validar live

Depende de: Cualquier plano nuevo con Derrotero.zip.

Descripción: Cuando se sube un Derrotero.zip, el validador compara área y coordenadas del shapefile con lo declarado en el cajetín. Si difieren >5% → error bloqueante. Falta validar el comportamiento con shapefiles malformados o de otra proyección.

Cómo probar:

- Subir un plano nuevo con Derrotero.zip válido — validar que pasa OK
- Probar con un ZIP sin .prj — validar advertencia
- Probar con coords en otra proyección (CR05 en vez de CRTM05)
- Probar con áreas que difieren >5% — validar error bloqueante

Componentes: `src/utils/derrotero_validator.py` · hook en `seed_builder.py`

Estado tests: ■ 13 tests + live test contra FELIPE_TIOS exitoso.

#10 Detección auto tipo BD (segregación/rectificación/etc.)

MEDIA — validar live

Depende de: Próximo plano nuevo, sin declarar tipo manualmente.

Descripción: El detector lee el texto del cajetín y del registro para decidir si es segregación, rectificación, reunión, info posesoria o finca completa. Validar contra casos reales.

Cómo probar:

- Para próximo plano, NO declarar tipo al crear expediente
- Correr el extractor Vision
- Verificar que `detectar_tipo_plano()` devuelve el tipo correcto
- Probar casos ambiguos (mezcla de palabras clave)

Componentes: `src/utils/tipo_plano_detector.py`

Estado tests: ■ 19 tests. Falta integración con flujo de creación de expediente.

#11 Memoria operador (APT REGLA / APT IGNORA)

MEDIA — validar live

Depende de: WhatsApp activo y comandos enviados por operador.

Descripción: Operador puede agregar reglas operativas y silenciar discrepancias repetitivas via WhatsApp. Validar persistencia y que las ignoras silencian las notificaciones.

Cómo probar:

- Enviar: APT REGLA Para planos en RIO CUARTO incluir mapa adjunto

- Verificar respuesta del bot con ID #X
- Listar: APT REGLAS
- Provocar una discrepancia con cédula 2-0440-0388 (Grace/Amalia)
- Enviar: APT IGNORA 2-0440-0388
- Provocar la misma discrepancia → verificar que NO notifica
- Verificar que sigue en metadata.apt_discrepancias_rnp (auditoría)
- Desactivar: APT OLVIDA <id>

Componentes: src/agents/whatsapp_commands.py · src/utils/memoria_operador.py · tabla apt_memoria_operador

Estado tests: ■ 18 tests. Falta validación con WhatsApp real.

#12 Comando WhatsApp DEBUG

MEDIA — validar live

Depende de: WhatsApp activo.

Descripción: Operador pide DEBUG <numero> y recibe dump del estado interno del bot: progreso bP1-bP7, trámite, discrepancias, anomalías, snapshots disponibles, reglas activas.

Cómo probar:

- Enviar: DEBUG RDF-2026-004
- Verificar respuesta con todos los campos esperados
- Probar con expediente sin contrato APT
- Probar con expediente enviado al CFIA
- Probar con expediente que tenga discrepancias/anomalías

Componentes: src/agents/whatsapp_commands.py (_handle_debug)

Estado tests: ■ Tests indirectos. Falta validación end-to-end con WhatsApp real.

#13 Email digest semanal

MEDIA — validar live

Depende de: Lunes a las 7am UTC con scheduler corriendo.

Descripción: El digest envía resumen del dashboard a los admins con correo configurado en usuarios.correo_apt. Validar que llega correctamente formateado.

Cómo probar:

- Configurar email de admin: UPDATE usuarios SET correo_apt='...' WHERE rol='admin'
- Forzar envío inmediato: python tools/catastro_bot.py enviar-digest
- Verificar que el correo llega y se ve bien (texto + HTML)
- Esperar al lunes 7am y validar que el scheduler lo dispara automáticamente

Componentes: src/utils/email_digest.py · scheduler tasks.py (próximo)

Estado tests: ■ 13 tests. Falta validación con SMTP real.

#14 2FA para acciones sensibles

MEDIA — validar live

Depende de: Integración con whatsapp_commands para acciones ENVIAR CFIA / RECHAZAR / borrado.

Descripción: El módulo TwoFactorAuth genera códigos 6-digit con TTL 5 min para autorizar acciones críticas. Falta integrarlo en los handlers de WhatsApp y luego validarlo en vivo.

Cómo probar:

- Integrar en _handle_rechazar / _handle_appt_enviar_cfia (TODO)
- Operador envía: ENVIAR CFIA RDF-2026-005
- Bot responde con código de 6 dígitos
- Operador envía: CONFIRMAR <codigo>
- Verificar que el bot ejecuta la acción
- Probar timeout: esperar 5+ min sin confirmar → código expira
- Probar código incorrecto 5 veces → invalidación por max_intentos

Componentes: src/utils/two_factor_auth.py · src/agents/whatsapp_commands.py (integración pendiente)

Estado tests: ■ 16 tests del módulo. Falta integración + validación live.

#15 Rate limiting WhatsApp (Green API)

MEDIA — validar live

Depende de: Bot enviando muchos mensajes simultáneamente (ej. discrepancia masiva).

Descripción: El RateLimiter previene throttling de Green API. Falta integrarlo en WhatsAppAgent.

Cómo probar:

- Integrar GREEN_API_LIMITER en WhatsAppAgent.enviar_mensaje (TODO)
- Provocar burst de 20 notificaciones simultáneas
- Verificar que se envían a ritmo controlado (~5/seg) sin throttling
- Revisar stats del limiter: total_acquired, total_waited_sec

Componentes: src/utils/rate_limiter.py · src/agents/whatsapp_agent.py (integración pendiente)

Estado tests: ■ 11 tests del módulo. Falta integración + carga real.

#16 Selector resilience (fallback por NAME)

BAJA — validar live

Depende de: Que APT cambie un selector / haga deploy nuevo.

Descripción: Si APT renombra un ID (#dllprotocolo → #ddlProtocolo), el bot debe seguir funcionando vía el fallback `[name='Profesional.Protocolo']`. Difícil de probar hasta que pase.

Cómo probar:

- Simular: monkey-patch un ID en una página de prueba → cambiarlo a otro
- Verificar que el bot usa el fallback automáticamente
- Revisar logs: 'usando fallback X' debería aparecer
- En producción: la primera vez que APT cambie algo, ver si el bot lo absorbe

Componentes: src/agents/appt_agent.py (FALLBACKS_CAMPOS_APT + _localizar_resilient)

Estado tests: ■ 11 tests. Falta evento real (cuando APT cambie).

#17 Snapshot post-mortem en anomalías

BAJA — validar live

Depende de: Que ocurra una anomalía real durante el llenado.

Descripción: Cuando APTAnomalyError dispara, el bot guarda HTML+PNG+modal+meta en data/anomaly_snapshots/. Falta validar que los snapshots son útiles para debugging post-mortem.

Cómo probar:

- Provocar una anomalía deliberada (modal de error)
- Inspeccionar data/anomaly_snapshots/<exp>/<ts>/
- Validar que page.html abre en navegador y muestra el estado real
- Validar que screenshot.png muestra lo visible al momento de fallar
- Validar que modal_actual.json captura el swal de error

Componentes: src/utils/anomaly_snapshot.py · src/agents/apt_anomaly_handler.py

Estado tests: ■ 13 tests con mocks. Falta validación con anomalía real.

#18 State machine + resume desde crash

BAJA — validar live

Depende de: Crash del bot a mitad del llenado del plano (improbable pero posible).

Descripción: Si el bot muere mientras llena el plano, al reiniciar debe leer apt_progreso + verificar estado live en APT, y retomar desde la sección siguiente sin reintentar las verdes.

Cómo probar:

- Llenar bP1+bP2+bP3 → verificar que apt_progreso se persiste
- Matar el proceso a mitad de bP4: taskkill /F /PID <pid>
- Re-correr el runner
- Verificar que muestra 'SKIP bP1', 'SKIP bP2', 'SKIP bP3' y empieza desde bP4

Componentes: src/utils/apt_progreso.py · tools/run_apt_plano_auto.py

Estado tests: ■ 19 tests. Falta simular crash real mid-llenado.

#19 Shadow mode con auto_if_clean

BAJA — validar live

Depende de: Validación de 10+ planos exitosos antes de subir trust.

Descripción: Cambiar BOT_SAVE_MODE_CONTRATO de 'manual' a 'auto_if_clean' una vez que tengamos confianza alta. El bot guardará automáticamente cuando no haya discrepancias.

Cómo probar:

- Después de 10 planos exitosos en modo manual, cambiar config:
- \$env:BOT_SAVE_MODE_CONTRATO='auto_if_clean'
- Correr un plano limpio → debe guardar automáticamente
- Correr un plano con discrepancia → debe pausar
- Volver a manual si algo sale mal: unset env

Componentes: src/utils/shadow_mode.py · config/settings.py · tools/run_apt_crear_auto.py

Estado tests: ■ 14 tests del shadow_mode. Falta uso en producción.

#20

Logs estructurados JSON

BAJA — validar live

Depende de: Necesidad de ingest a herramientas externas (ELK, jq).

Descripción: Cambiar formato de logs a JSON line para facilitar parsing con jq/log-aggregators.

Cómo probar:

- Setear: `$env:CATASTRO_LOG_FORMAT='json'`
- Reiniciar el bot
- Correr cualquier acción
- Verificar logs en `logs/catastro-bot.log` → cada línea es JSON válido
- Probar: `Get-Content logs/catastro-bot.log | jq '.level="ERROR"'`

Componentes: `src/utils/logger.py` (JsonFormatter)

Estado tests: ■ 10 tests. Falta uso real con herramientas externas.

#21

CLI unificado catastro-bot

BAJA — validar live

Depende de: Uso diario por el operador.

Descripción: Un solo entry point reemplaza los ~25 `tools/*.py`. Falta que el operador lo adopte como hábito y reporte qué subcomandos faltan o tienen UX mejorable.

Cómo probar:

- Reemplazar todos los alias / scripts batch que usen `tools/*.py` por `catastro-bot <sub>`
- Documentar en `CLAUDE.md` o `README` la sintaxis nueva
- Probar autocompletado bash/PowerShell (opcional)
- Recoger feedback del operador después de 1 semana de uso

Componentes: `tools/catastro_bot.py`

Estado tests: ■ 4 tests del router. Falta validación de UX.

Resumen de capas de defensa del bot

1. Vision auto-extract → llena seed desde PDFs/PNGs
2. Seed builder + advertencias → marca datos dudosos
3. Pre-flight (sin red) → bloquea errores estructurales antes de Chrome
4. Auto-firma SSO si cert BCR activo → sesión APT sin re-PIN
5. Resume desde progreso saved+live → skip secciones ya verdes
6. Multi-topógrafo → per-user protocolo y correo
7. Shadow mode → controla qué se auto-guarda
8. Memoria operador → silencia falsos positivos persistentemente
9. Selector resilience → sobrevive cambios de ID en APT
10. Strict modal validation → APTAnomalyError si APT rechaza
11. Circuit breaker → MessageBox + WhatsApp + snapshot
12. Healthcheck + watchdog → detecta Chrome caído y relanza
13. Backup BD diario → protección contra corrupción/pérdida
14. Rate limiting → previene throttling Green API
15. 2FA acciones sensibles → confirma envío/rechazo con código
16. Dashboard + alertas proactivas → visibilidad operativa
17. Email digest semanal → resumen sin abrir WhatsApp
18. DEBUG WhatsApp → self-service del operador
19. CLI unificado → un solo entry point para todo
20. Logs JSON → ingest a herramientas externas